🧪 Práctica integradora de vectores — 1

Monitoreo de un aerogenerador

Cátedra: Programación en Computación — UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

---

> 🎯 **Esta práctica tiene el formato del 1er parcial:** un caso de ingeniería, cinco incisos, y el primero genera los datos. Resolvela como si fuera el examen: todo en funciones, con parámetros y valor de retorno.
>
> **No hace falta saber de energía eólica.** Todo lo que hay que calcular está definido abajo con su fórmula.

📘 Contexto

Un parque eólico monitorea uno de sus aerogeneradores de **2000 kW**. Un anemómetro en la góndola registra la **velocidad del viento** (en m/s) una vez por hora durante un día: **24 mediciones**, que se guardan en un **vector V**.

Cuánta potencia entrega el aerogenerador depende de la velocidad del viento, según esta **curva de potencia**:

VELOCIDAD DEL VIENTO	ESTADO	POTENCIA ENTREGADA (KW)
menos de 3 m/s	`PARADO`	0 — no hay viento suficiente para arrancar
de 3 a menos de 12 m/s	`PARCIAL`	crece con el viento (fórmula del inciso 2)
de 12 a 25 m/s	`NOMINAL`	2000 — su máximo
más de 25 m/s	`CORTE`	0 — se frena para no romperse

Definí los datos fijos como **constantes** al principio del programa:

```
const HOURS = 24
const RATED_POWER = 2000   // kW
const CUT_IN = 3           // m/s
const RATED_SPEED = 12     // m/s
const CUT_OUT = 25         // m/s
const GUST_LIMIT = 8       // m/s
```

Se pide

1. Generación de datos

Escribí una función que genere un vector de **enteros aleatorios** dentro de un rango, y usala para generar el vector **V** de 24 velocidades entre **0 y 26 m/s**. Reusá `rndInt()`.

```
const getRandomVector = (size, min, max) => { ... }
```

> 💡 Los valores son aleatorios. **Para poder comparar con los ejemplos de salida**, mientras desarrollás reemplazá la llamada por este vector fijo — todos los ejemplos lo usan:
>
> ```
> const V = [2, 4, 6, 9, 11, 13, 15, 14, 12, 10, 19, 24,
>            27, 17, 16, 13, 9, 7, 5, 3, 1, 4, 8, 6]
> ```

2. Potencia y energía

Escribí una función que calcule la **potencia** para **una** velocidad de viento, y otra que genere el **vector P** con la potencia de cada hora:

```
si v < 3  o  v > 25:     P = 0
si 3 <= v < 12:          P = 2000 * ((v - 3) / 9) ** 3        redondeada a 1 decimal
si 12 <= v <= 25:        P = 2000
```

```
const powerAt = (speed) => { ... }
const getPowerVector = (speeds) => { ... }
```

Con el vector P, calculá la **energía generada en el día** y el **factor de capacidad**, que dice qué porcentaje de su máximo posible generó el aerogenerador:

```
energía (kWh)          = P[0] + P[1] + ... + P[23]        (cada medición vale una hora)
energía (MWh)          = energía (kWh) / 1000
factor de capacidad    = energía (kWh) / (2000 * 24) * 100
```

Ejemplo de salida:

```
Potencia por hora P (kW):
[
       0,   2.7,  74.1, 592.6,
  1404.7,  2000,  2000,  2000,
    2000,   941,  2000,  2000,
       0,  2000,  2000,  2000,
   592.6, 175.6,  21.9,     0,
       0,   2.7, 342.9,  74.1
]

Energía generada en el día: 22.22 MWh
Factor de capacidad: 46.3 %
```

> ⚠️ Fijate en la hora 12: con **27 m/s** la potencia es **0**, no 2000. El aerogenerador está en corte. Es el error más fácil de cometer en este inciso.

3. Estados de operación

Escribí una función que devuelva el **estado** para **una** velocidad, según la tabla del contexto, y otra que genere el **vector C** con el estado de cada hora:

```
si v < 3:              "PARADO"
si 3 <= v < 12:        "PARCIAL"
si 12 <= v <= 25:      "NOMINAL"
si v > 25:             "CORTE"
```

```
const stateAt = (speed) => { ... }
const getStateVector = (speeds) => { ... }
const countState = (states, state) => { ... }    // cuántas horas en un estado
```

Mostrá cuántas horas estuvo en cada estado.

Ejemplo de salida:

```
Horas por estado:
  PARADO:  2
  PARCIAL: 12
  NOMINAL: 9
  CORTE:   1
```

4. Ráfagas

Una **ráfaga** es una hora en la que el viento cambió **más de 8 m/s** respecto de la hora anterior, subiendo o bajando:

```
cambio[i] = V[i] - V[i - 1]

hay ráfaga en la hora i   si   cambio[i] > 8   o   cambio[i] < -8
```

Escribí una función que **cuente** las ráfagas del día, y mostrá en qué hora ocurrió cada una y de cuánto fue el cambio.

```
const countGusts = (speeds) => { ... }
```

Ejemplo de salida:

```
Ráfagas detectadas: 2
  Hora 10: 10 → 19 m/s (+9)
  Hora 13: 27 → 17 m/s (-10)
```

> ⚠️ La hora 0 **no tiene hora anterior**. ¿Desde qué índice tiene que arrancar el bucle?

5. Reporte del día

Escribí las funciones necesarias para encontrar:

- La **hora de mayor potencia**: la primera hora en la que P es máxima.
- La **primera hora en corte**: la primera hora con estado `"CORTE"`, o **-1** si en todo el día no hubo ninguna.

```
const peakHour = (power) => { ... }                 // devuelve la hora
const firstHourIn = (states, state) => { ... }      // devuelve la hora, o -1
```

El programa tiene que interpretar ese `-1` y mostrar un mensaje distinto en cada caso.

Ejemplo de salida:

```
── Reporte del día ──
Hora de mayor potencia: 5 (2000 kW con 13 m/s)
Primer corte por viento excesivo: hora 12 (27 m/s)
```

Si en el día no hubo corte:

```
No hubo cortes por viento excesivo
```

⛔ Restricciones

- ❌ **Sin** `map`, `filter`, `reduce`, `forEach`, `find`, `sort`, `indexOf`, `push`, `fill`.

- ❌ **Sin** retornar objetos: sólo números, textos, booleanos y vectores.
- ✅ Todo en **funciones** con parámetros y valor de retorno, con su **JSDoc**.
- ✅ Toda función que devuelve un vector lo devuelve con `.slice()`.
- ✅ Los vectores se crean con `new Array(n)` y se llenan con un bucle.

> 🔍 **Antes de usar una función en el programa, probala en la consola de Node** con casos difíciles: `powerAt(3)`, `powerAt(25)`, `powerAt(26)`. Los bordes de los rangos son donde están los errores.

📎 Material de consulta

- Presentación de la unidad 07 (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/presentacion.pdf)
- Apunte de la unidad 07 (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/apunte.pdf)
- Template del curso (https://github.com/italijancic/pc-2026/tree/main/template)